Esecuzione delle operazioni

- I DBMS offrono operatori che implementano ciascuno uno o più operatori dell'algebra
- Operatori fondamentali:
  - scansione
  - accesso diretto
- A livello più alto:
  - ordinamento
- Ancora più alto
  - **join**

Join

- L'operazione più costosa
- Vari metodi; i più noti:
  - *nested-loop*, *merge-join* and *hash-join*

Nested-loop

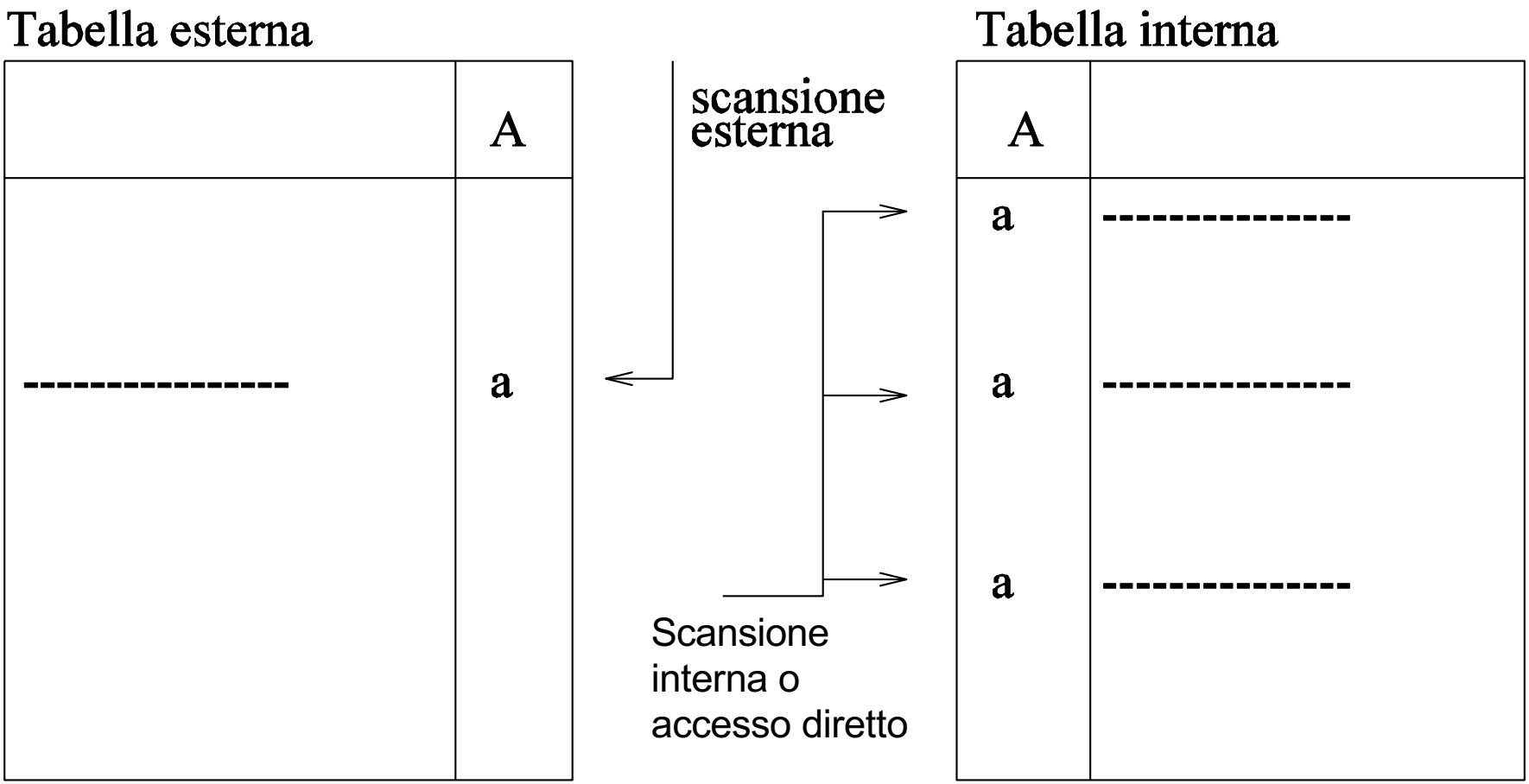
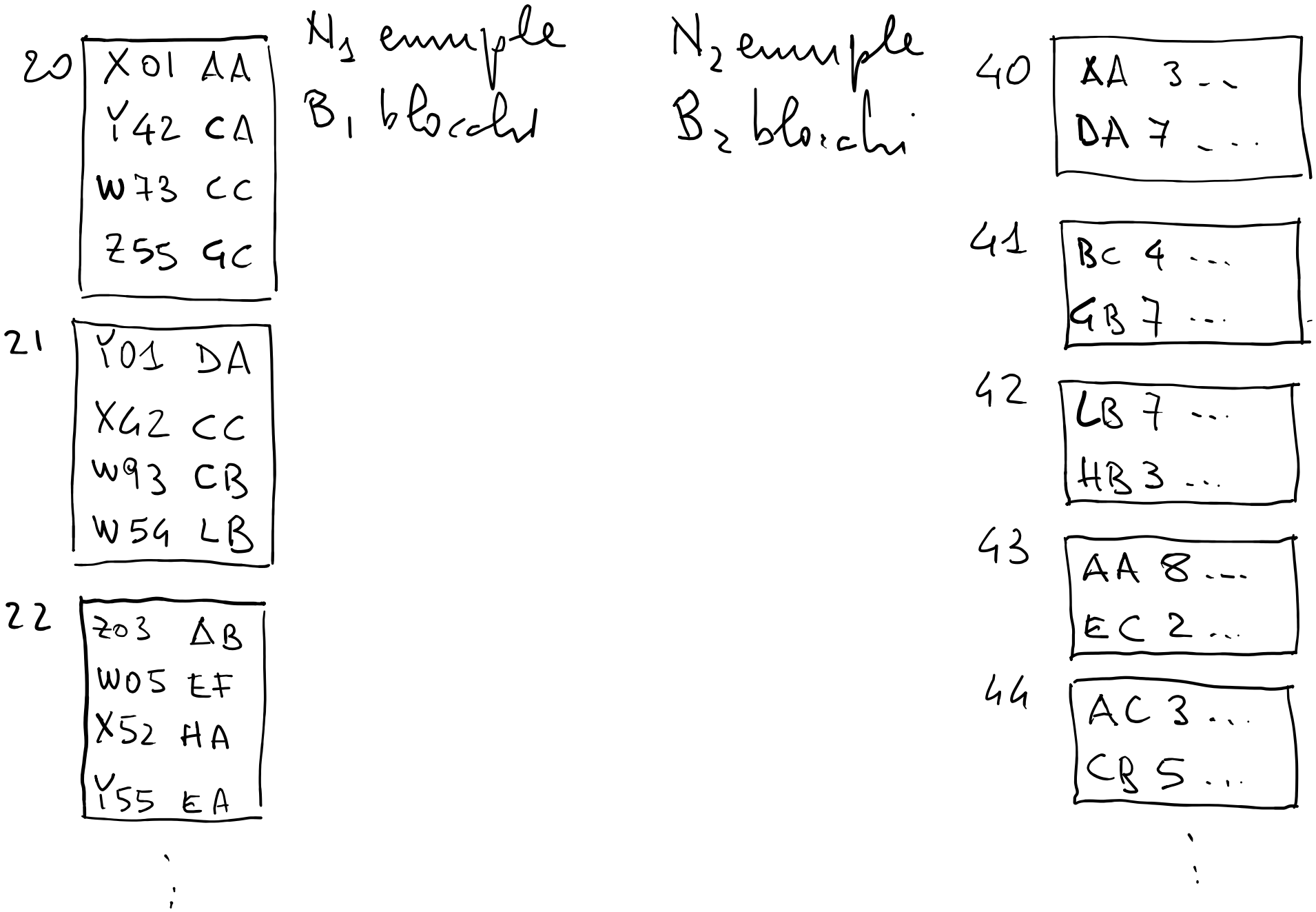
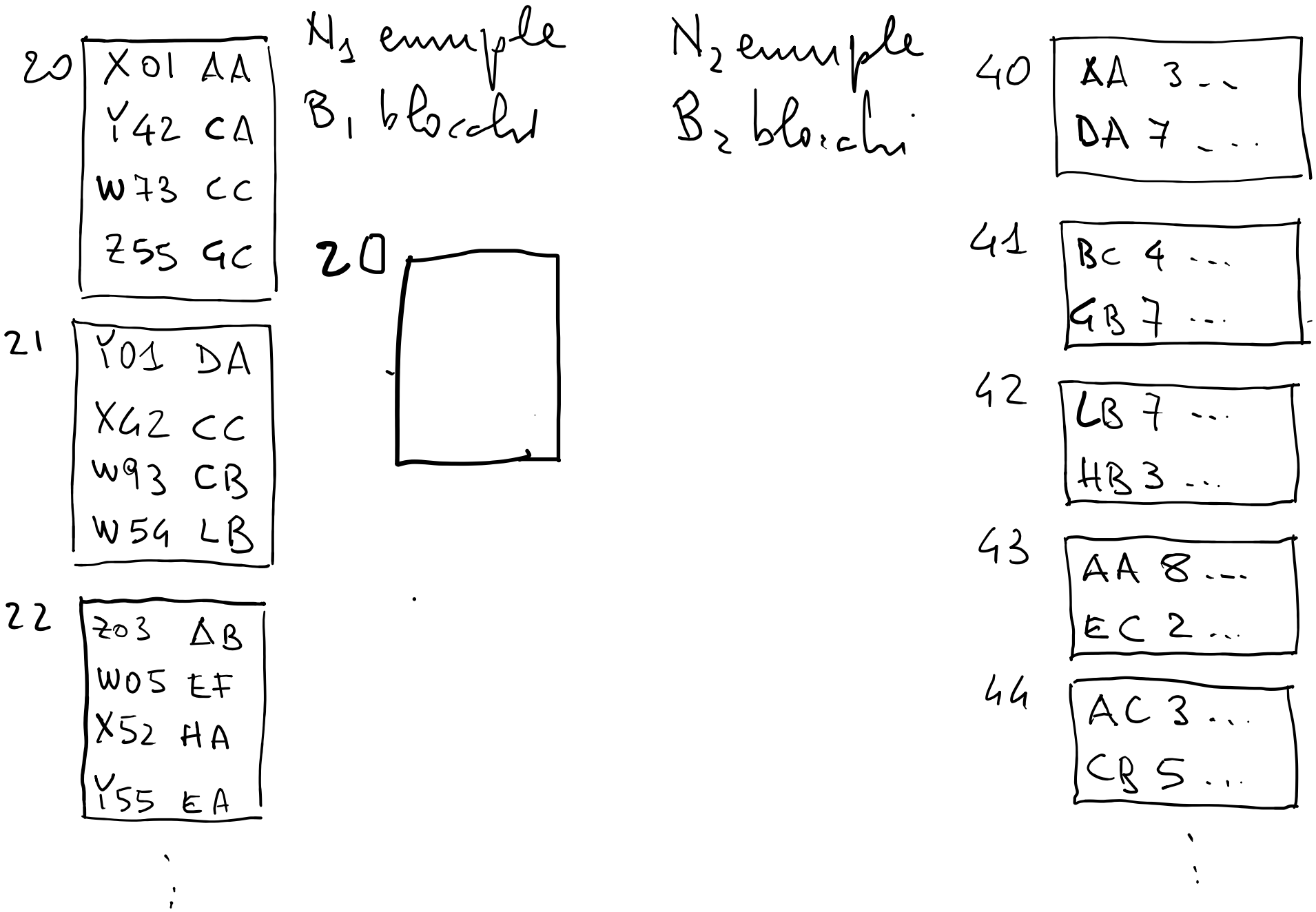


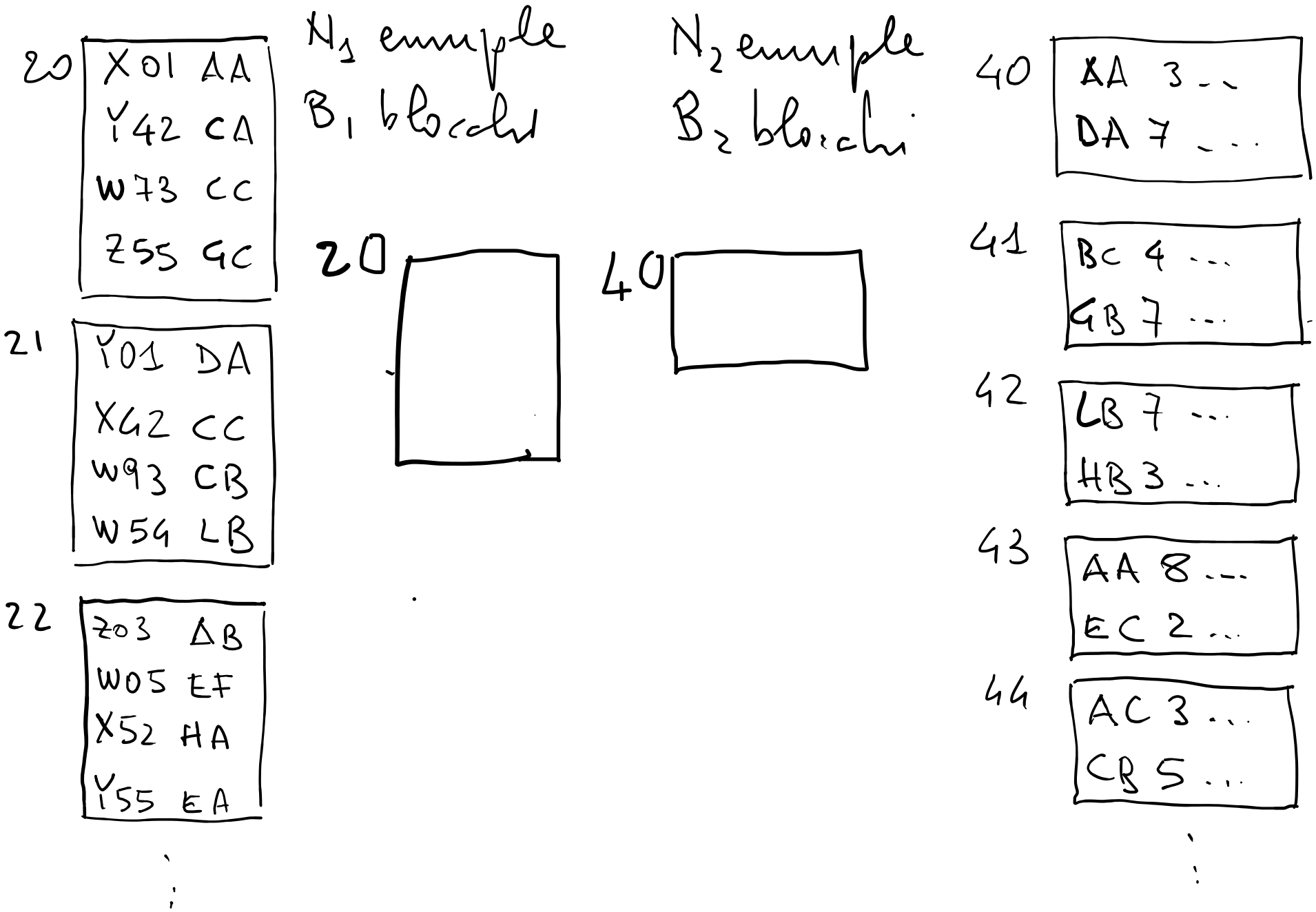
Tabella esterna
A
a
scansione esterna
Tabella interna
A
a
a
a
Scansione interna o accesso diretto

Nested loop, senza indice

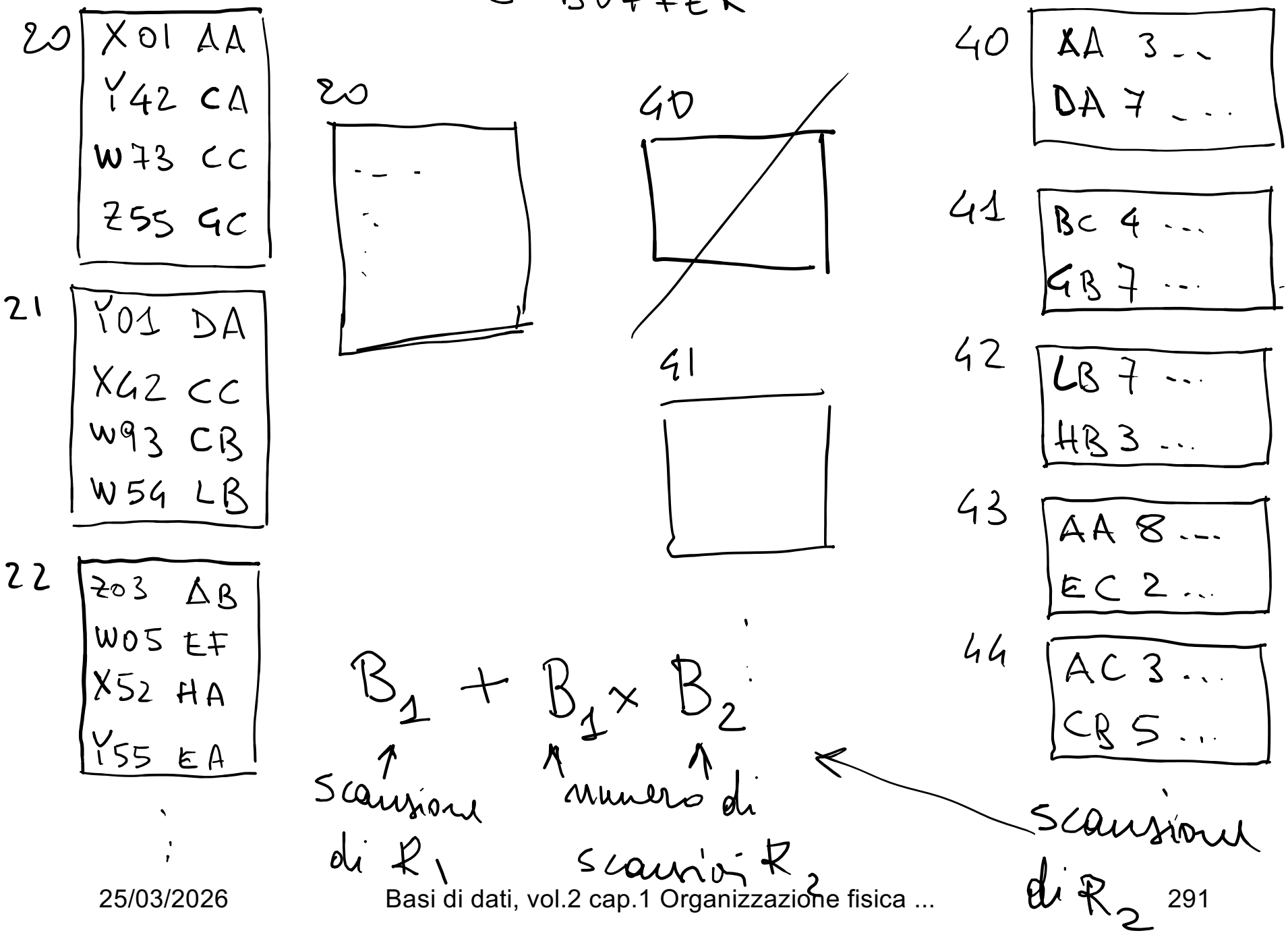
- Alla lavagna
  - scansione delle ennuple
  - scansione dei blocchi
  - utilizzo dei buffer


N1 ennple
B1 blocchi
N2 ennple
B2 blocchi
20
X01 AA
Y42 CA
W73 CC
Z55 GC
21
Y01 DA
X42 CC
W93 CB
W54 LB
22
Z03 AB
W05 EF
X52 HA
Y55 EA
40
AA 3 ...
DA 7 ...
41
BC 4 ...
GB 7 ...
42
LB 7 ...
HB 3 ...
43
AA 8 ...
EC 2 ...
44
AC 3 ...
CB 5 ...


N1 esempio
B1 blocchi
N2 esempio
B2 blocchi
20
X01 AA
Y42 CA
W73 CC
Z55 GC
21
Y01 DA
X42 CC
W93 CB
W54 LB
22
Z03 AB
W05 EF
X52 HA
Y55 EA
20
40
AA 3 ...
DA 7 ...
41
BC 4 ...
GB 7 ...
42
LB 7 ...
HB 3 ...
43
AA 8 ...
EC 2 ...
44
AC 3 ...
CB 5 ...


N1 ennple
B1 blocchi
N2 ennple
B2 blocchi
20
X01 AA
Y42 CA
W73 CC
Z55 GC
21
Y01 DA
X42 CC
W93 CB
W54 LB
22
Z03 AB
W05 EF
X52 HA
Y55 EA
20
40
40
AA 3 ...
DA 7 ...
41
BC 4 ...
GB 7 ...
42
LB 7 ...
HB 3 ...
43
AA 8 ...
EC 2 ...
44
AC 3 ...
CB 5 ...

2 BUFFER



$$B_1 + B_1 \times B_2$$

scansione di R_1

numero di scansioni R_2

scansione di R_2

Nested-loop, costi

- Join, con nested loop, senza indici;
    - relazioni R_1 e R_2 di N_1 e N_2 blocchi
  - algoritmo base:
    - scansione di R_1 e, per ciascun blocco di R_1, scansione di R_2; costo (numero di accessi a memoria secondaria):

$$N_1 + N_1 \times N_2$$

Costo

$$B_1 + B_1 \times B_2$$

scansione

n. scansioni

scansione

Come posso sfruttare i bufffer?

Costo

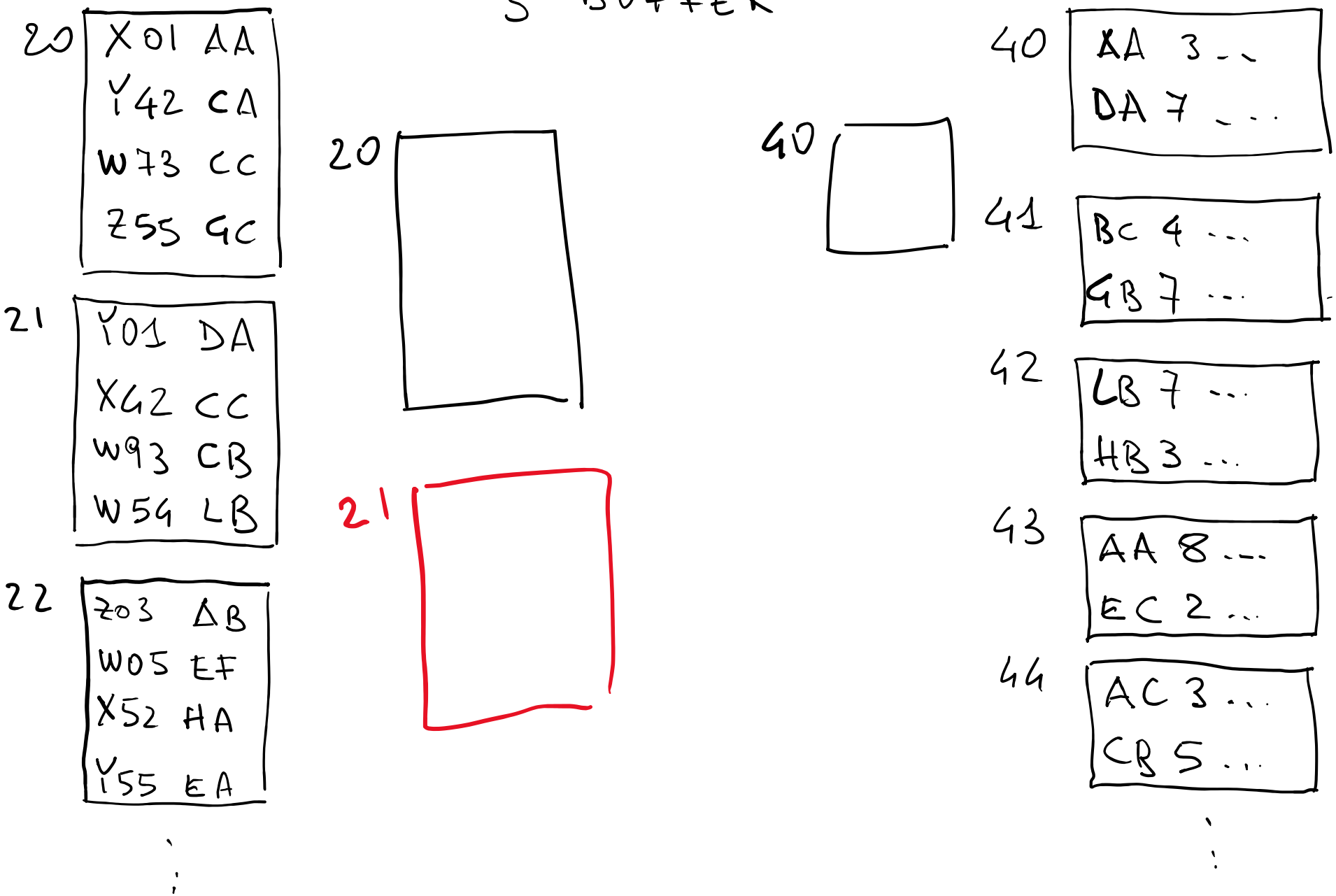
$$B_1 + B_1 \times B_2$$

scansione

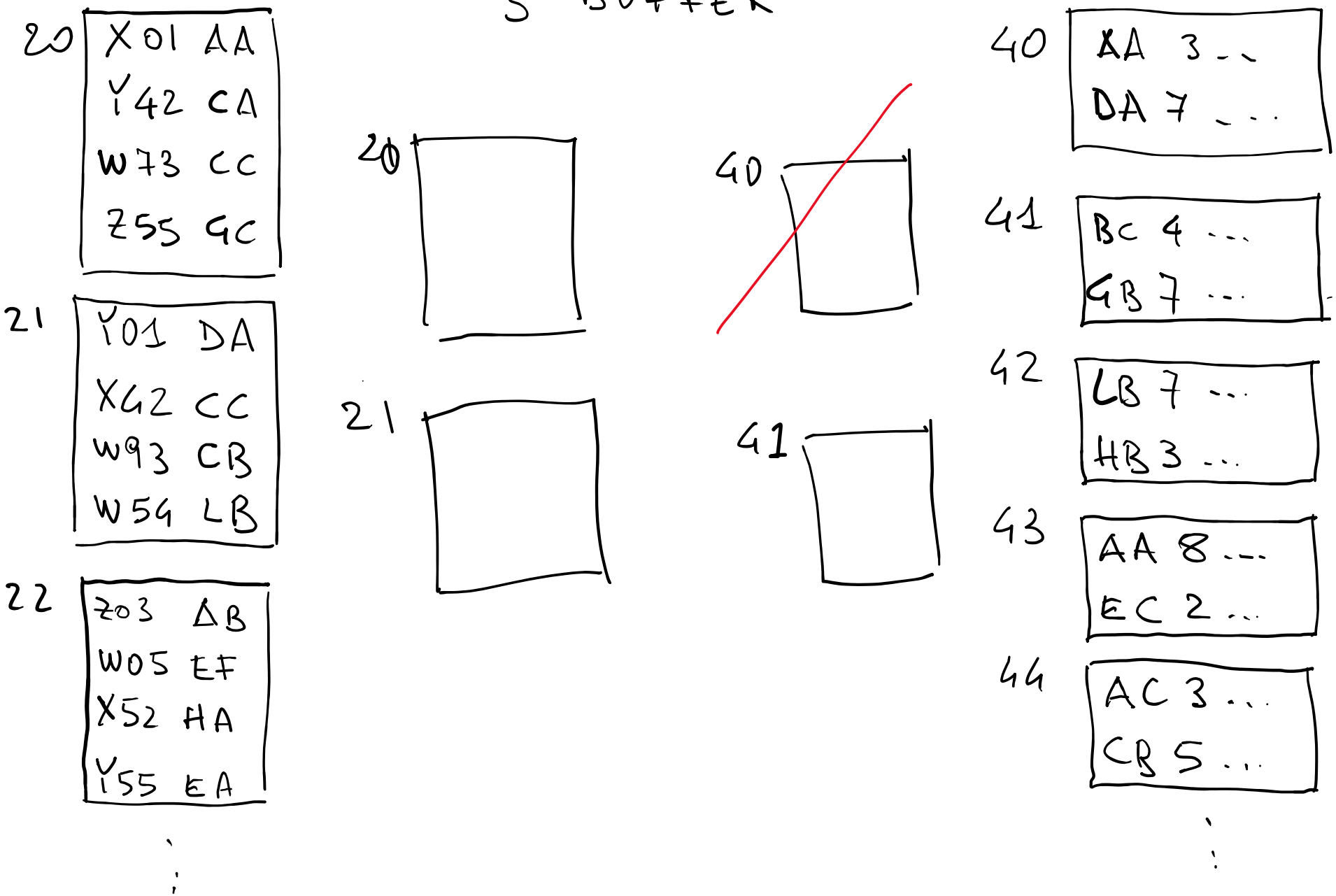
n. scansioni

scansione

3 BUFFER


20
X01 AA
Y42 CA
W73 CC
Z55 GC
21
Y01 DA
X42 CC
W93 CB
W54 LB
22
Z03 AB
W05 EF
X52 HA
Y55 EA
20
21
40
40
AA 3 ...
DA 7 ...
41
BC 4 ...
GB 7 ...
42
LB 7 ...
HB 3 ...
43
AA 8 ...
EC 2 ...
44
AC 3 ...
CB 5 ...

3 BUFFER


20
X01 AA
Y42 CA
W73 CC
Z55 GC
21
Y01 DA
X42 CC
W93 CB
W54 LB
22
Z03 AB
W05 EF
X52 HA
Y55 EA
20
21
40
41
40
AA 3 ...
DA 7 ...
41
BC 4 ...
GB 7 ...
42
LB 7 ...
HB 3 ...
43
AA 8 ...
EC 2 ...
44
AC 3 ...
CB 5 ...

Nested-loop, costi

- Join, con nested loop, senza indici;
  - relazioni R_1 e R_2 di N_1 e N_2 blocchi
  - algoritmo base:
    - scansione di R_1 e, per ciascun blocco di R_1, scansione di R_2; costo (numero di accessi a memoria secondaria):

$$N_1 + N_1 \times N_2$$

  - con più pagine di buffer
    - carichiamo più blocchi di R_1, riducendo il numero di scansioni di R_2 (le ennuple di R_2 durante la scansione possono essere confrontate con quelle in tutti i blocchi di R_1 nel buffer); con B pagine di buffer dedicate a blocchi di R_1 il costo diventa

$$N_1 + (N_1/B \times N_2)$$

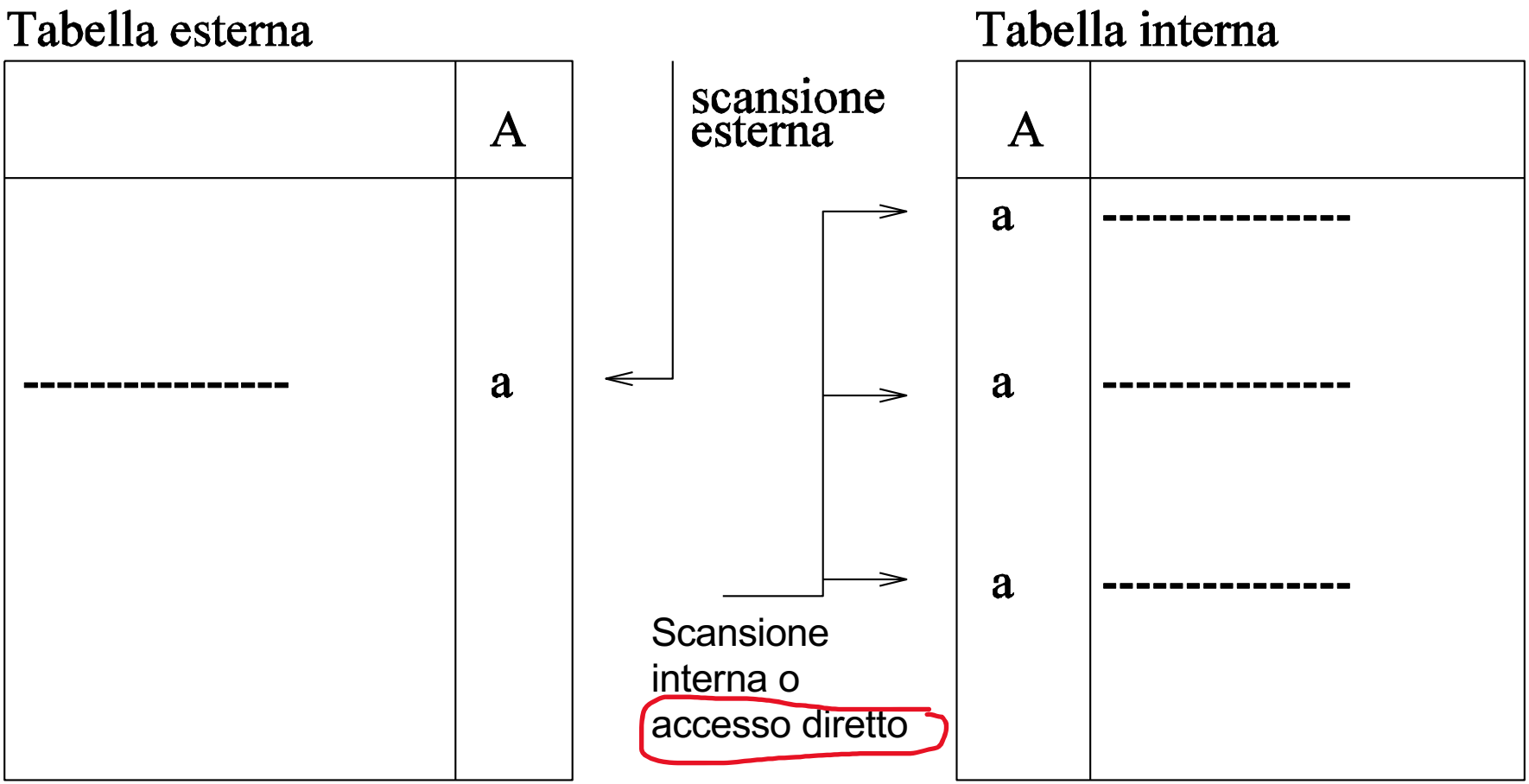
Nested-loop, costi (con buffer!)

- Join, con nested loop
  - R1: N1=1000 blocchi
  - R2: N2=500 blocchi
  - P=101 pagine a disposizione nel buffer

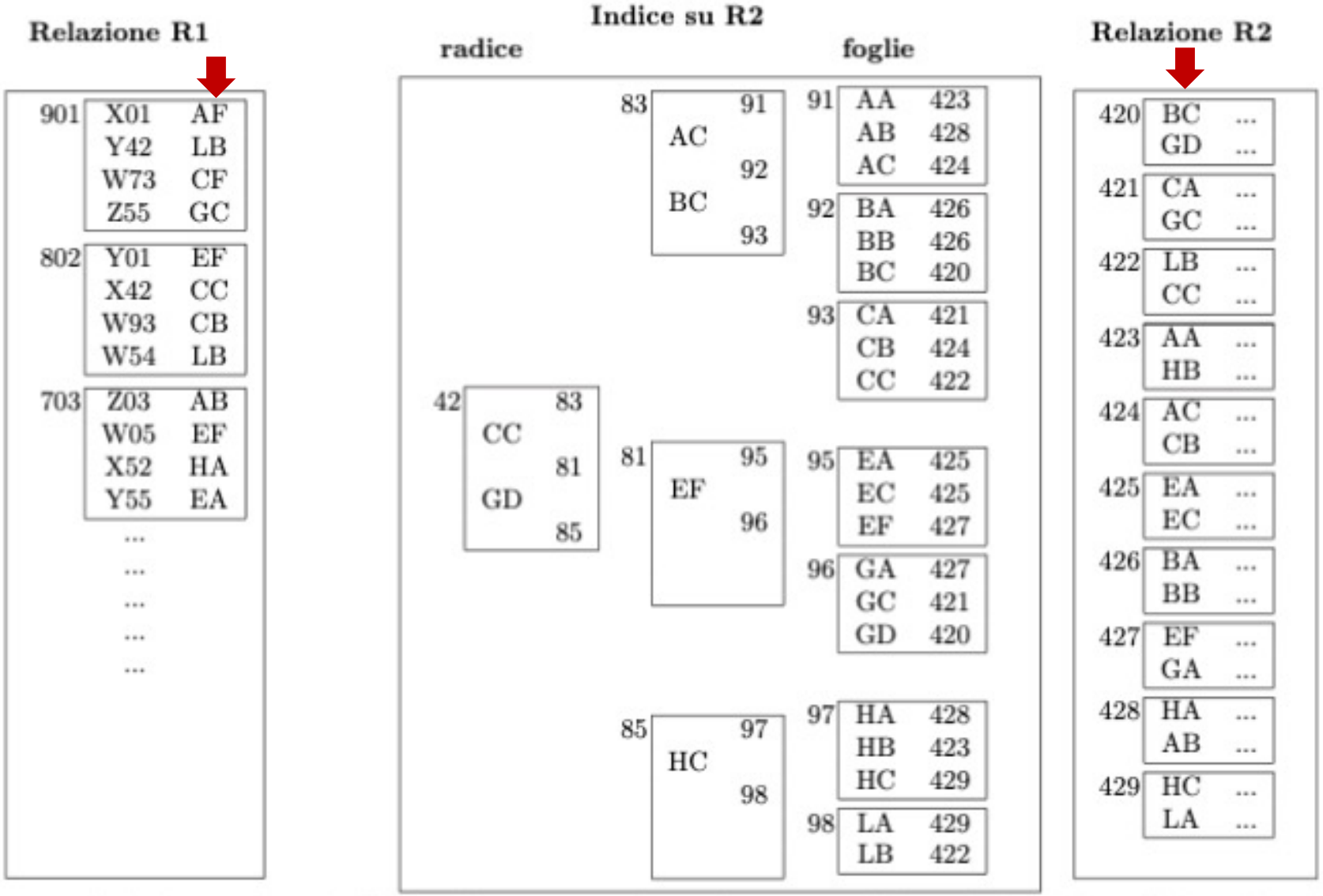
  - $N1 + N1/(P-1) \times N2 = 1000 + 10 \times 500 = \text{ca } 6000$

- Senza sfruttare buffer grandi (quindi leggendo un solo blocco per relazione) abbiamo 500.000 accessi,
- Con buffer circa 6.000

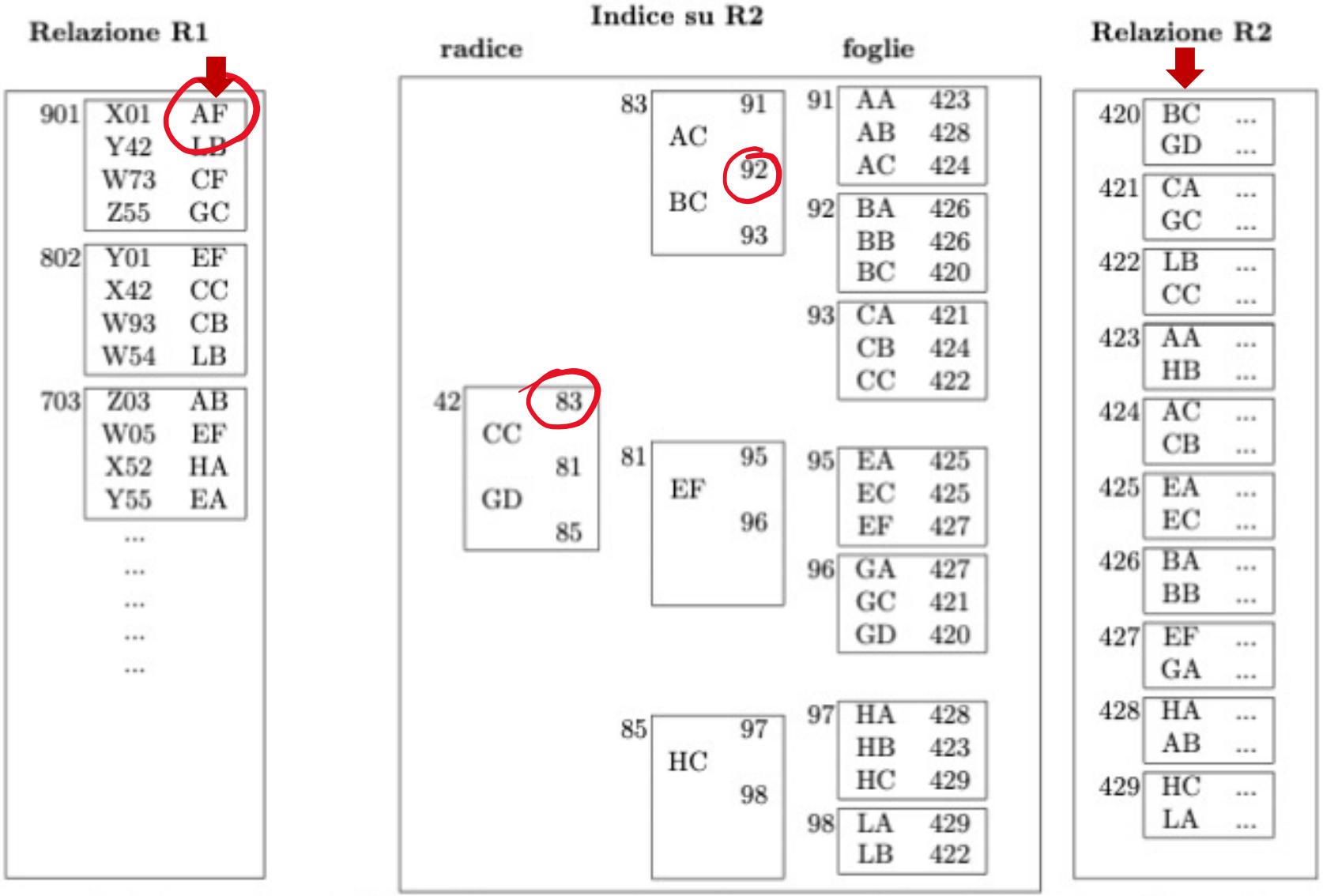
Nested-loop con indice



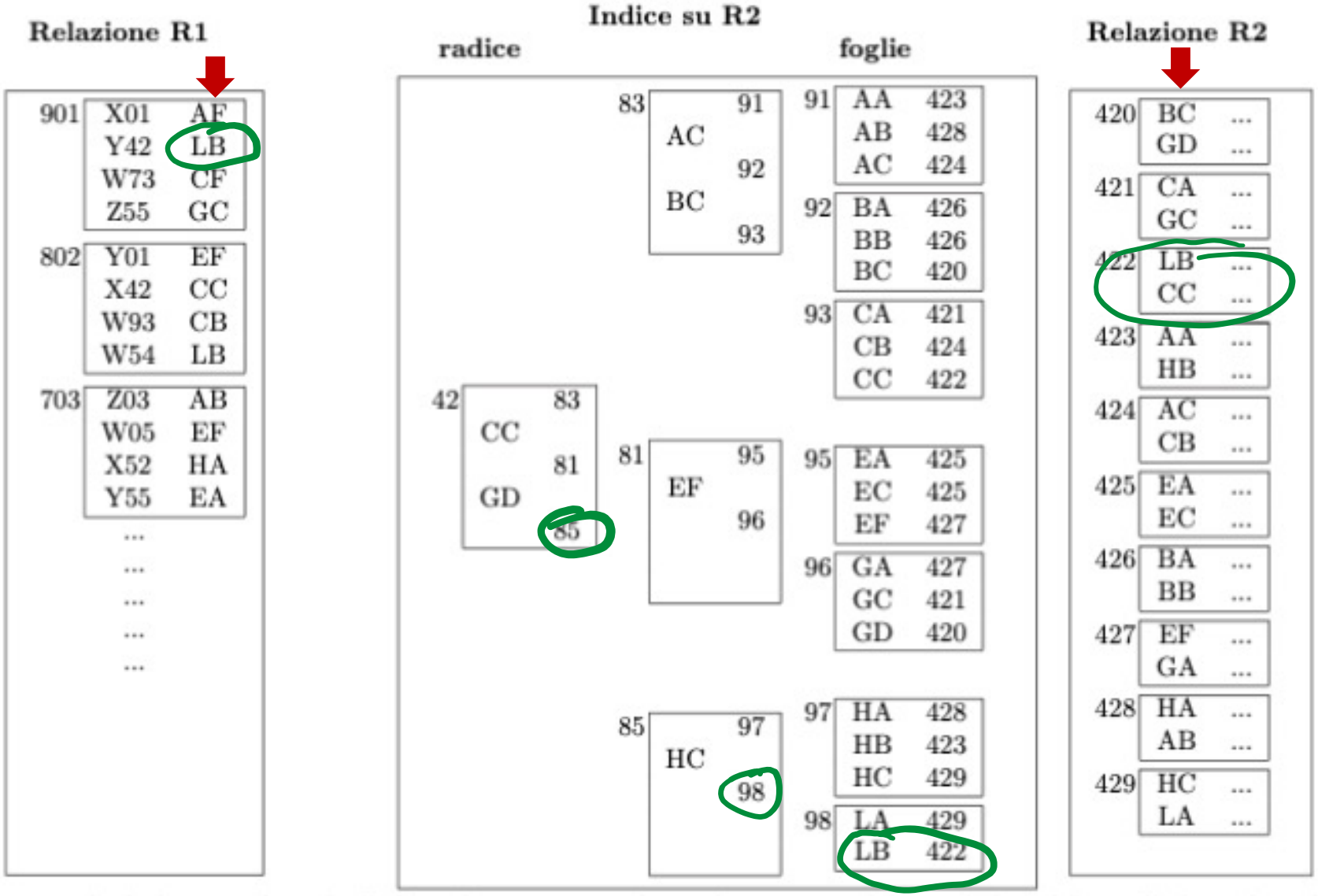
Nested loop, con indice



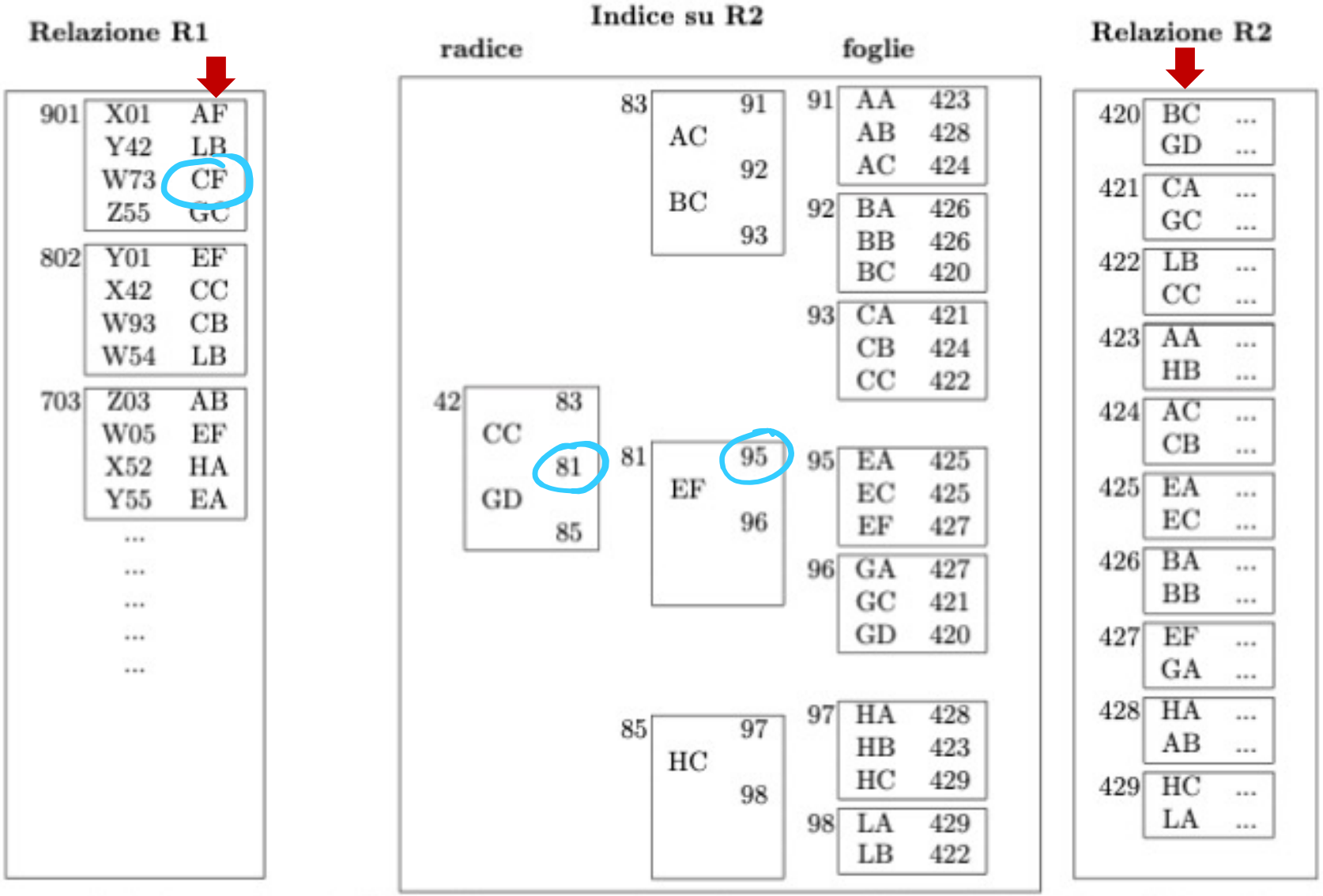
Nested loop, con indice



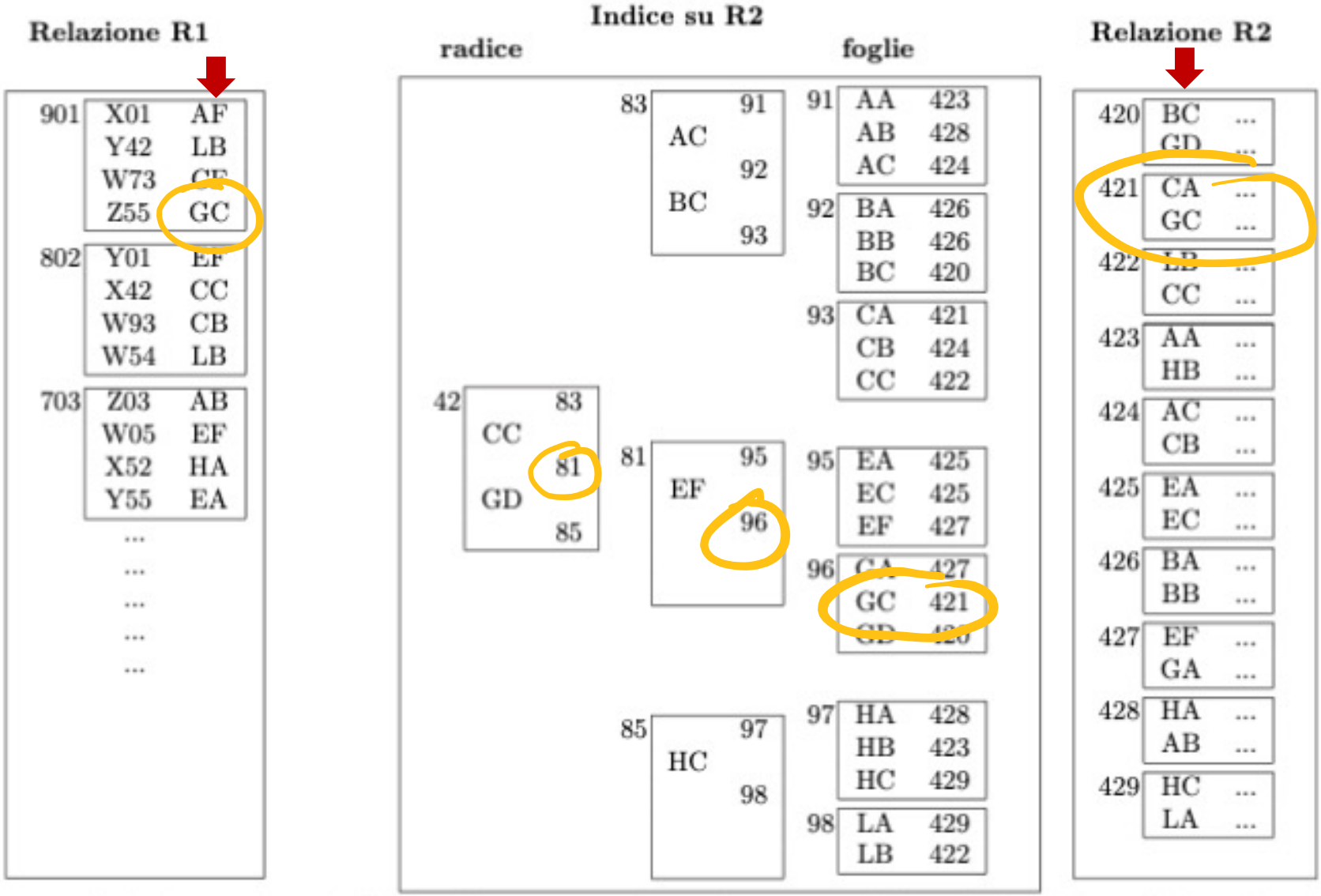
Nested loop, con indice



Nested loop, con indice



Nested loop, con indice



Nested-loop con indice, costi

- Join, con nested loop, con indice
  - R_1 con L_1 ennuple e N_1 blocchi
  - R_2 con L_2 ennuple e N_2 blocchi e indice di profondità I_2
  - l'algoritmo base:
    - scansione di R_1 e, per ciascun record di R_1, accesso diretto a R_2; costo:

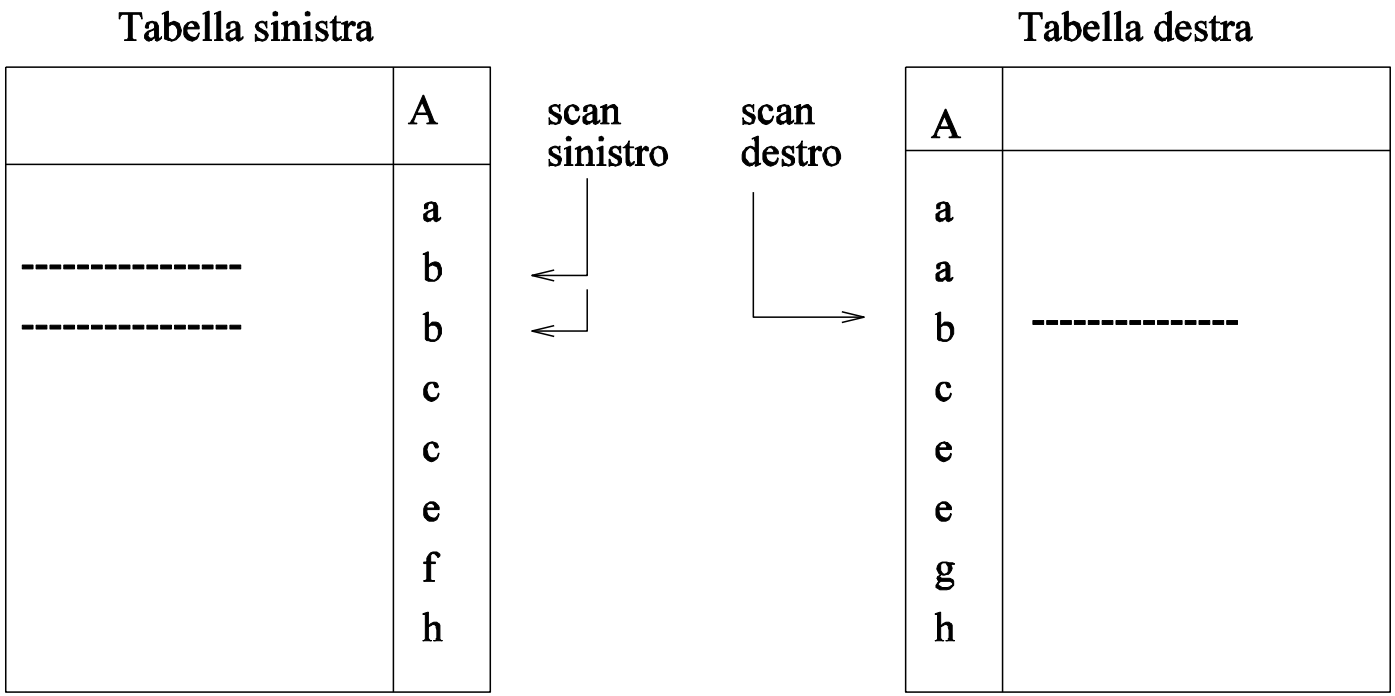
$$N_1 + L_1 \times (I_2 + 1)$$

  - con più pagine di buffer, i livelli più alti dell'indice, ad esempio due, sono nel buffer (perché ripetutamente usati durante il join)

$$N_1 + L_1 \times (I_2 - 2 + 1)$$

Merge-join

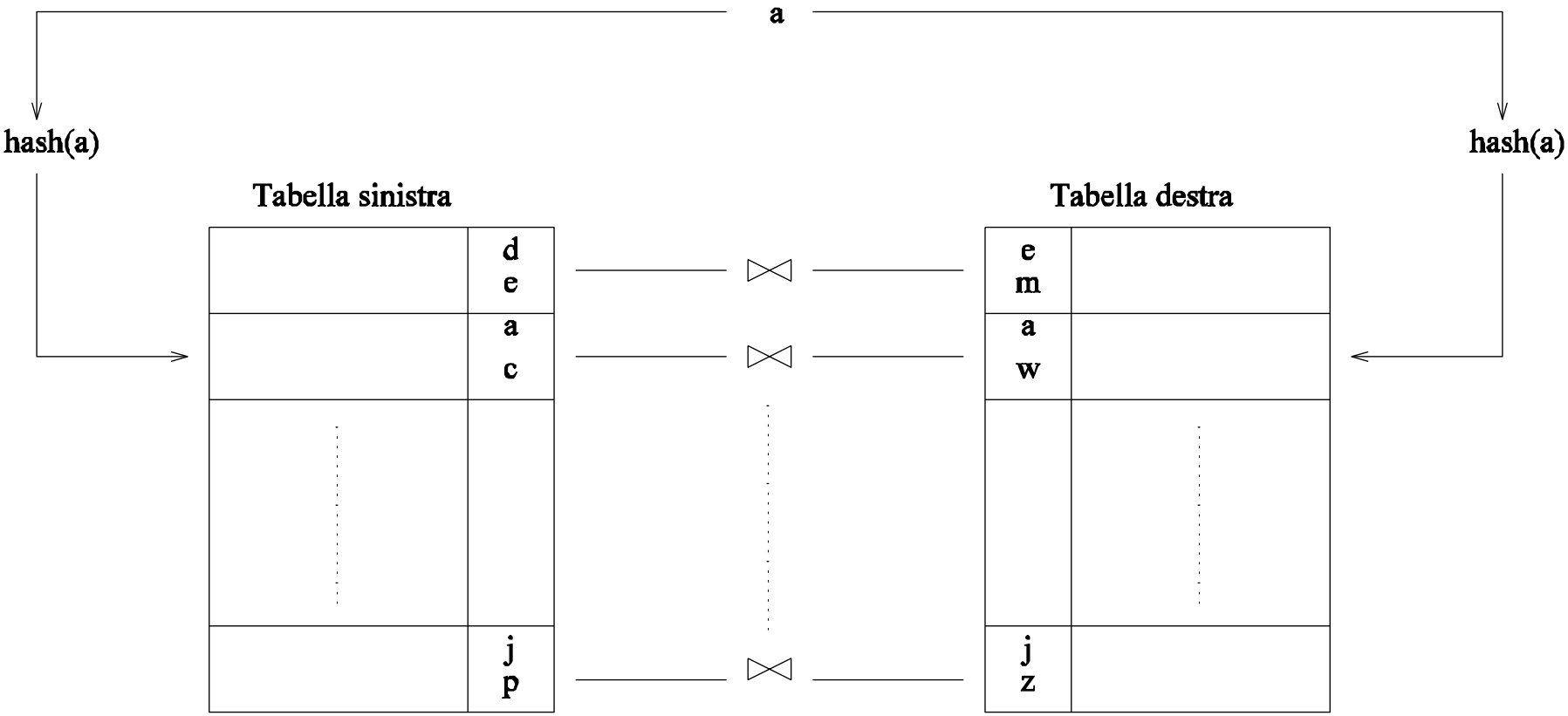
- Intuizione:
  - se entrambi I file sono ordinati sul campo di join, possiamo fare due scansioni parallele



Merge-join, costi

- Approssimativamente (N1 e N2 numero di blocchi dei due file):
  - se i file sono fisicamente ordinati:
    - N1 + N2
  - se sono disordinati e li ordiniamo preliminarmente (o meglio, durante il join stesso):
    - costo dei due ordinamenti (supponendo due passate):
      - 3 × (N1 + N2)
  - se i file sono disordinati, ma ci sono indici:
    - L1 + L2 (se tutti i record partecipano; se invece il join è selettivo il costo diminuisce) più il costo della scansione delle foglie degli indici

Hash join



- Stessa funzione hash applicata ai campi di join delle due relazioni
  - NB: non c'è e non serve struttura hash sui due file

Hash join

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	902

N	907
P	903

Q	902
R	905

S	904
T	910

901	X
910	Y

918	X
911	Z

902	Z
912	Y

919	X
903	W

913	X
904	Y

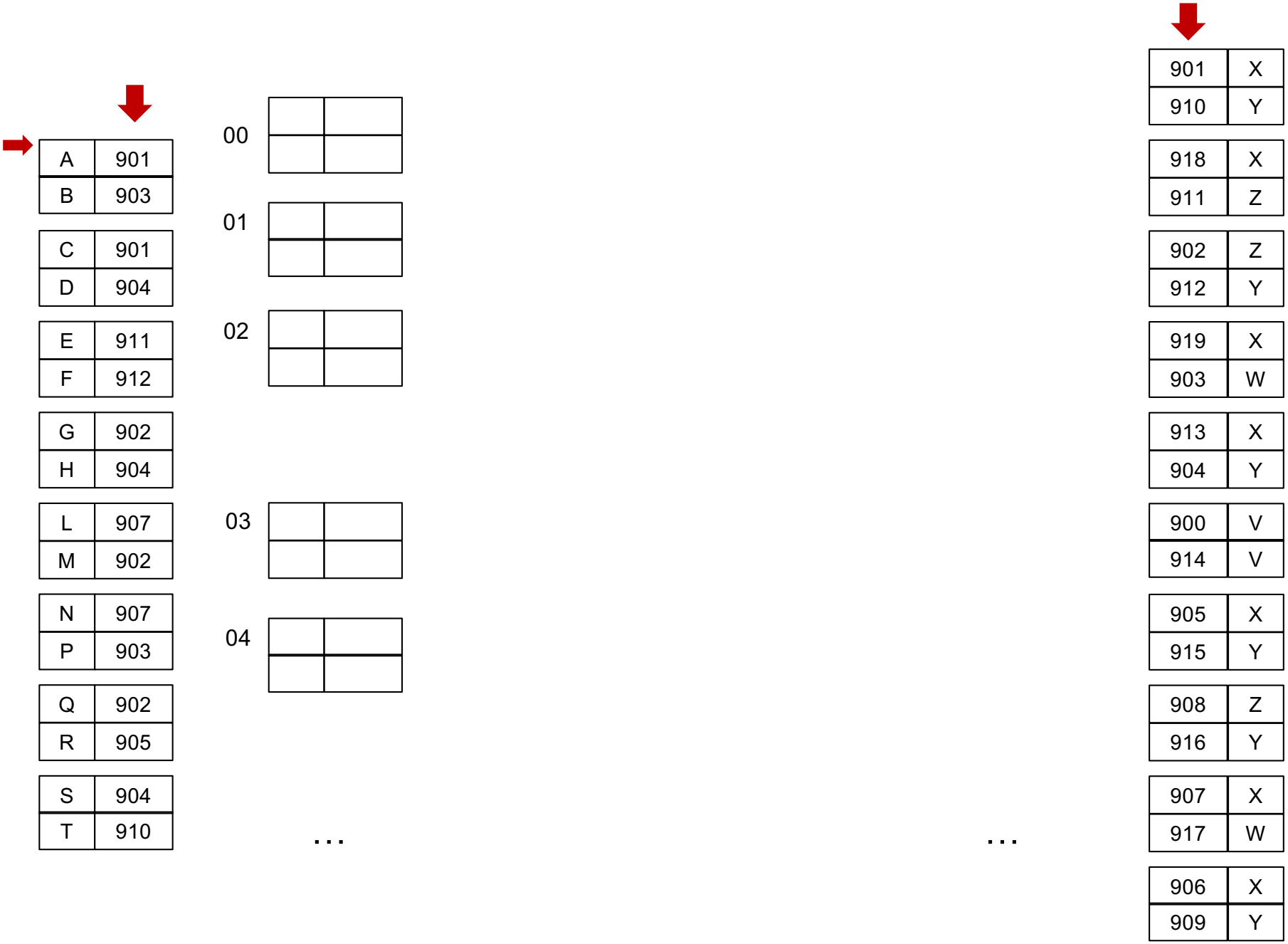
900	V
914	V

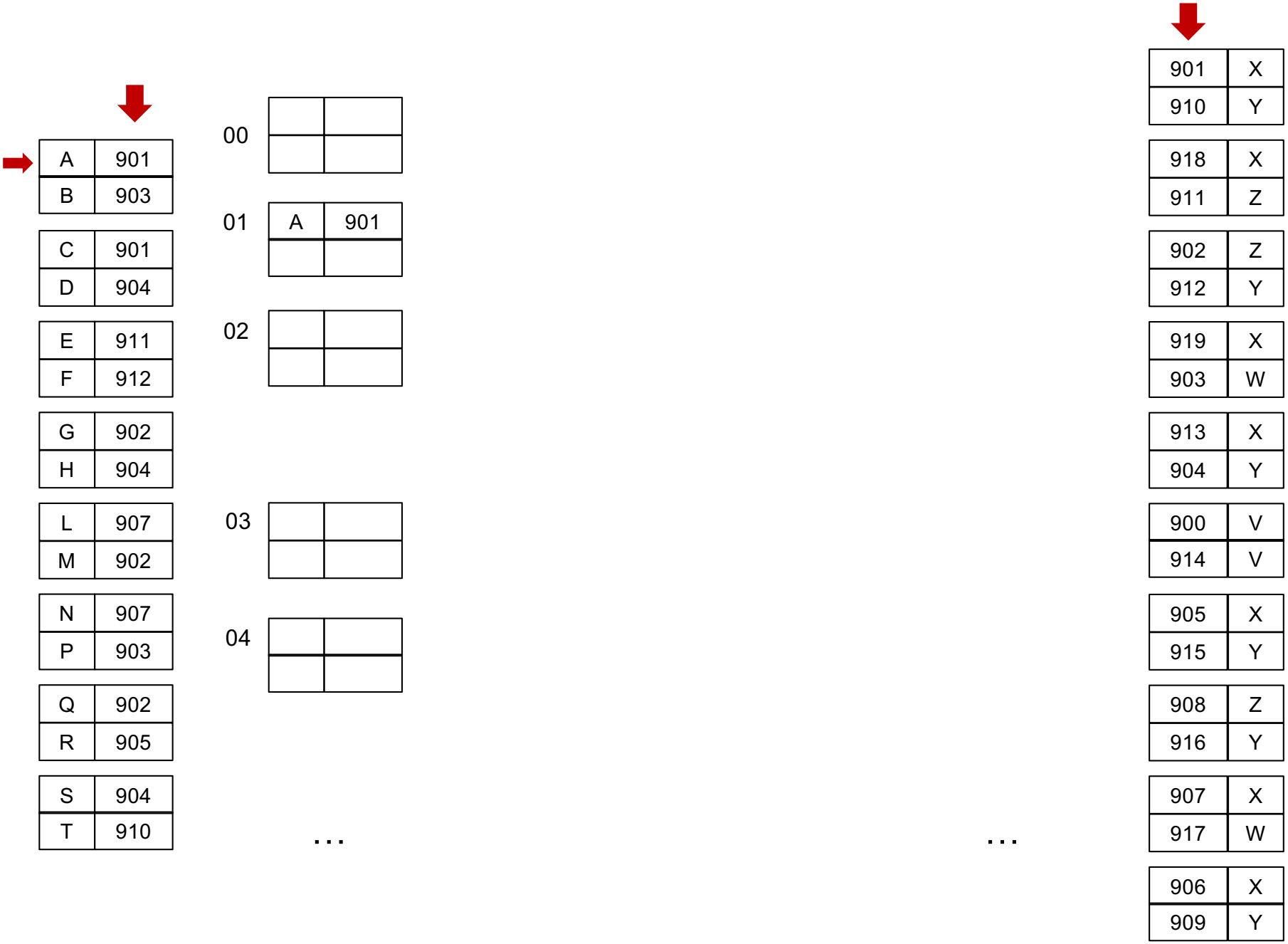
905	X
915	Y

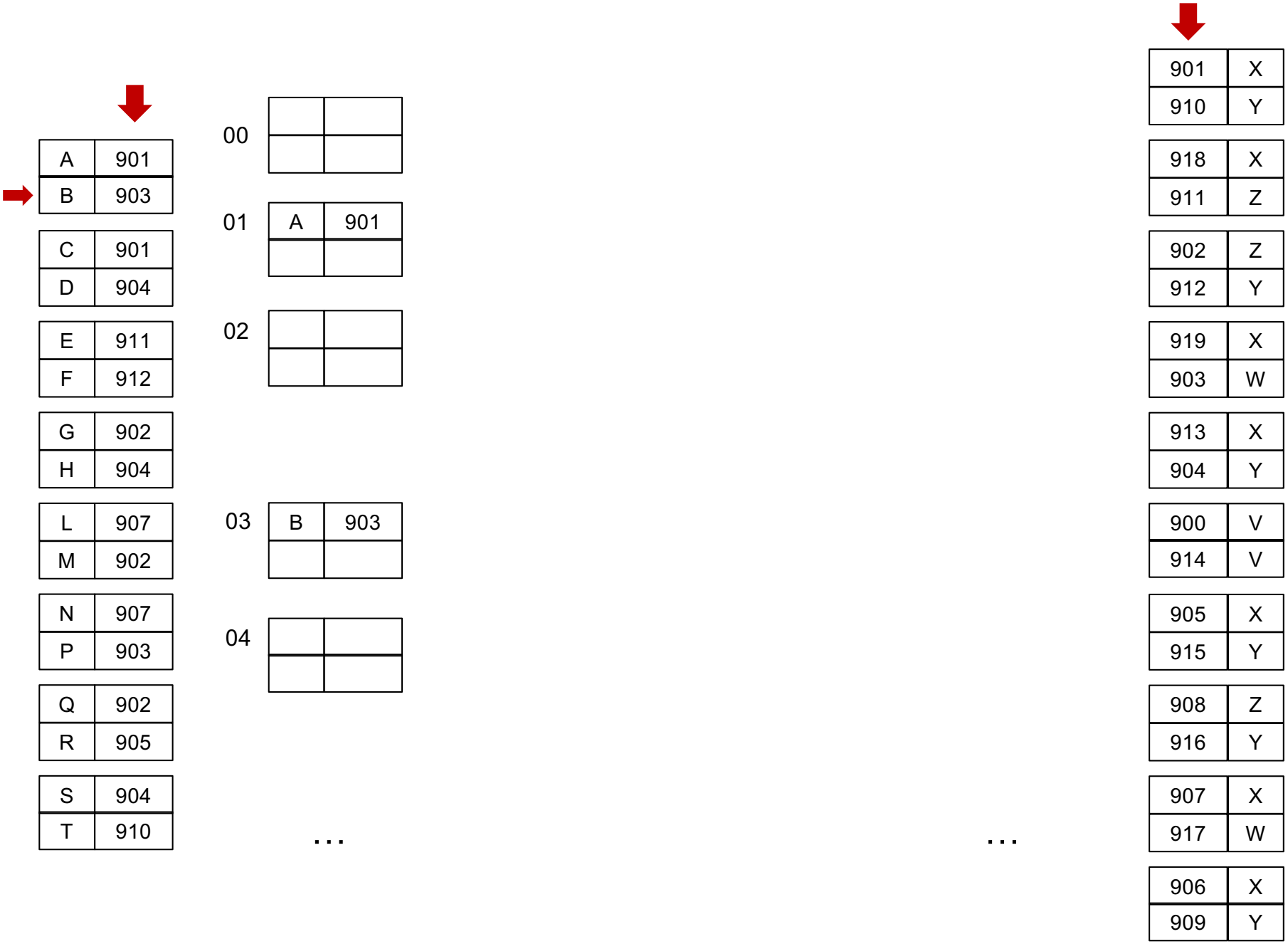
908	Z
916	Y

907	X
917	W

906	X
909	Y


A 901
B 903
C 901
D 904
E 911
F 912
G 902
H 904
L 907
M 902
N 907
P 903
Q 902
R 905
S 904
T 910
00
01
02
03
04
...
...
901 X
910 Y
918 X
911 Z
902 Z
912 Y
919 X
903 W
913 X
904 Y
900 V
914 V
905 X
915 Y
908 Z
916 Y
907 X
917 W
906 X
909 Y


A 901
B 903
C 901
D 904
E 911
F 912
G 902
H 904
L 907
M 902
N 907
P 903
Q 902
R 905
S 904
T 910
00
01
A 901
02
03
04
...
...
901 X
910 Y
918 X
911 Z
902 Z
912 Y
919 X
903 W
913 X
904 Y
900 V
914 V
905 X
915 Y
908 Z
916 Y
907 X
917 W
906 X
909 Y


A 901
B 903
C 901
D 904
E 911
F 912
G 902
H 904
L 907
M 902
N 907
P 903
Q 902
R 905
S 904
T 910
00
01 A 901
02
03 B 903
04
...
...
901 X
910 Y
918 X
911 Z
902 Z
912 Y
919 X
903 W
913 X
904 Y
900 V
914 V
905 X
915 Y
908 Z
916 Y
907 X
917 W
906 X
909 Y

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	902

N	907
P	903

Q	902
R	905

S	904
T	910

00

01

A	901
C	901

02

G	902
M	902

Q	902

03

B	903
P	903

04

D	904
H	904

S	904

...

...

901	X
910	Y

918	X
911	Z

902	Z
912	Y

919	X
903	W

913	X
904	Y

900	V
914	V

905	X
915	Y

908	Z
916	Y

907	X
917	W

906	X
909	Y

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	902

N	907
P	903

Q	902
R	905

S	904
T	910

00

01

A	901
C	901

02

G	902
M	902

Q	902

03

B	903
P	903

04

D	904
H	904

S	904

...

00

900	V

01

901	X

02

902	Z

03

903	W

04

904	Y

...

901	X
910	Y

918	X
911	Z

902	Z
912	Y

919	X
903	W

913	X
904	Y

900	V
914	V

905	X
915	Y

908	Z
916	Y

907	X
917	W

906	X
909	Y

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	902
N	907
P	903
Q	902
R	905
S	904
T	910

00		
01	A	901
	C	901
02	G	902
	M	902
	Q	902
03	B	903
	P	903
04	D	904
	H	904
	S	904

...

00	900	V
01	901	X
02	902	Z
03	903	W
04	904	Y

...

901	X
910	Y
918	X
911	Z
902	Z
912	Y
919	X
903	W
913	X
904	Y
900	V
914	V
905	X
915	Y
908	Z
916	Y
907	X
917	W
906	X
909	Y

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	902
N	907
P	903
Q	902
R	905
S	904
T	910

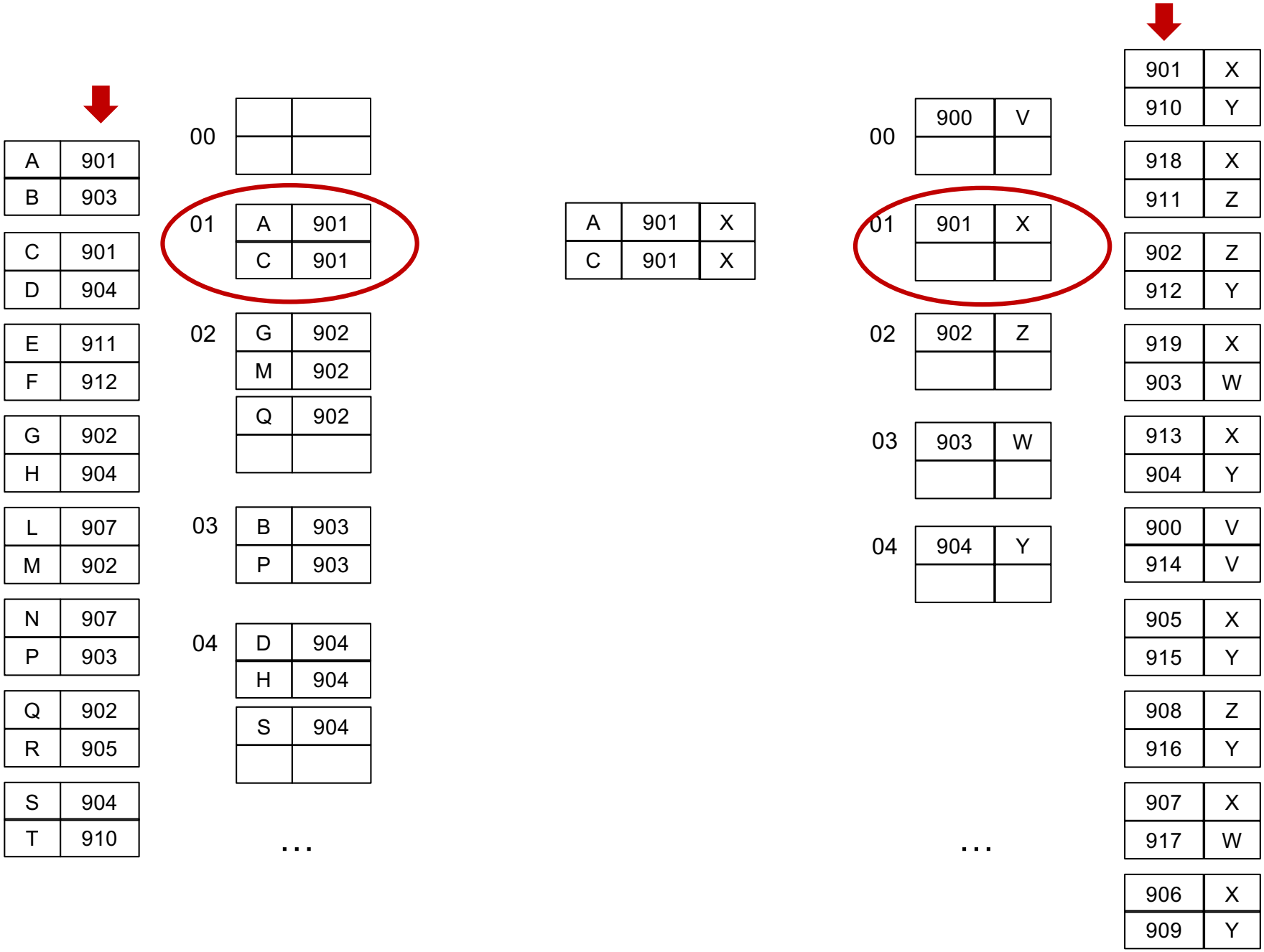
00		
01	A	901
	C	901
02	G	902
	M	902
	Q	902
03	B	903
	P	903
04	D	904
	H	904
	S	904

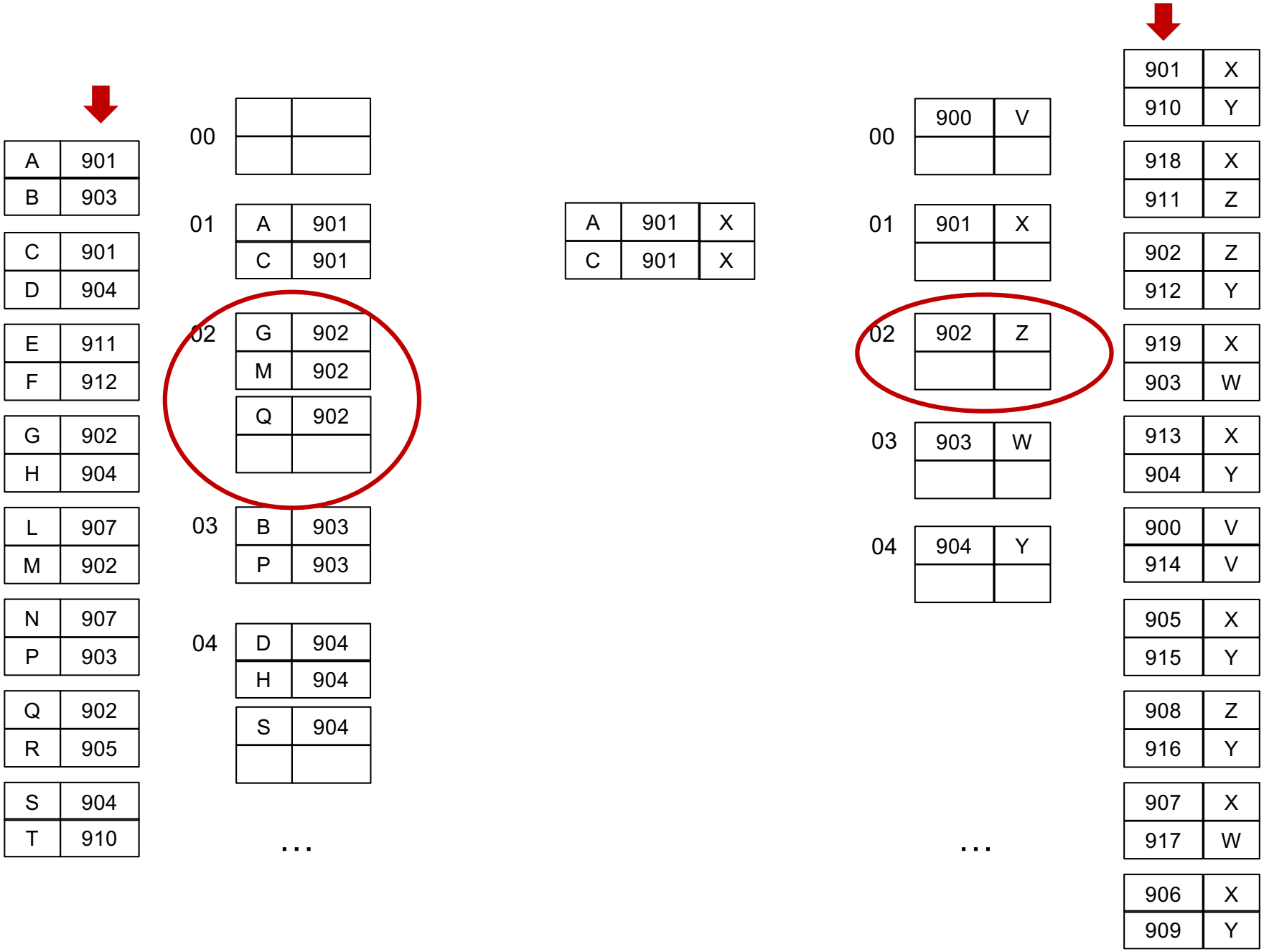
...

00	900	V
01	901	X
02	902	Z
03	903	W
04	904	Y

...

901	X
910	Y
918	X
911	Z
902	Z
912	Y
919	X
903	W
913	X
904	Y
900	V
914	V
905	X
915	Y
908	Z
916	Y
907	X
917	W
906	X
909	Y


A 901
B 903
C 901
D 904
E 911
F 912
G 902
H 904
L 907
M 902
N 907
P 903
Q 902
R 905
S 904
T 910
00
01 A 901 C 901
02 G 902 M 902 Q 902
03 B 903 P 903
04 D 904 H 904 S 904
...
A 901 X
C 901 X
00 900 V
01 901 X
02 902 Z
03 903 W
04 904 Y
...
901 X
910 Y
918 X
911 Z
902 Z
912 Y
919 X
903 W
913 X
904 Y
900 V
914 V
905 X
915 Y
908 Z
916 Y
907 X
917 W
906 X
909 Y


A 901
B 903
C 901
D 904
E 911
F 912
G 902
H 904
L 907
M 902
N 907
P 903
Q 902
R 905
S 904
T 910
00
01 A 901 C 901
02 G 902 M 902 Q 902
03 B 903 P 903
04 D 904 H 904 S 904
...
A 901 X
C 901 X
00 900 V
01 901 X
02 902 Z
03 903 W
04 904 Y
...
901 X
910 Y
918 X
911 Z
902 Z
912 Y
919 X
903 W
913 X
904 Y
900 V
914 V
905 X
915 Y
908 Z
916 Y
907 X
917 W
906 X
909 Y

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	902
N	907
P	903
Q	902
R	905
S	904
T	910

00		
01	A	901
	C	901
02	G	902
	M	902
	Q	902
03	B	903
	P	903
04	D	904
	H	904
	S	904

...

A	901	X
C	901	X
G	902	Z
M	902	Z
Q	902	Z
B	903	W

00	900	V
01	901	X
02	902	Z
03	903	W
04	904	Y

...

901	X
910	Y
918	X
911	Z
902	Z
912	Y
919	X
903	W
913	X
904	Y
900	V
914	V
905	X
915	Y
908	Z
916	Y
907	X
917	W
906	X
909	Y

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	902

N	907
P	903

Q	902
R	905

S	904
T	910

00

01

A	901
C	901

02

G	902
M	902

Q	902

03

B	903
P	903

04

D	904
H	904

S	904

...

A	901	X
C	901	X
G	902	Z
M	902	Z
Q	902	Z
B	903	W
P	903	W
D	904	Y
H	904	Y
S	904	Y

...

00

900	V

01

901	X

02

902	Z

03

903	W

04

904	Y

...

901	X
910	Y

918	X
911	Z

902	Z
912	Y

919	X
903	W

913	X
904	Y

900	V
914	V

905	X
915	Y

908	Z
916	Y

907	X
917	W

906	X
909	Y

Hash-join, primi commenti

- File con N1 e N2 record e B1 e B2 blocchi
- Con un approccio semplice (migliorabile, come vedremo):
  - primo file viene letto sequenzialmente (costo B1);
  - i record, durante la lettura, vengono memorizzati secondo la funzione hash (costo N1)
  - secondo file viene letto sequenzialmente (costo B2);
  - i record, durante la lettura, vengono memorizzati secondo la funzione hash (costo N2)
  - poi si rilegge il tutto (B1 e B2) per "accoppiare"
    - B1 + N1 + B2 + N2 + B1 + B2
- Ma sfruttando i buffer si può fare molto meglio

Hash-join con buffer

- Sfruttando i buffer, si può usare una funzione hash con numero di valori diversi paragonabile al numero di pagine di buffer P a disposizione

Hash join

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

901	X
910	Y

918	X
911	Z

902	Z
912	Y

919	X
903	W

913	X
904	Y

920	V
914	V

905	X
915	Y

908	Z
916	Y

907	X
917	W

906	X
909	Y

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

01

A	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

B	903

01

A	901

02

Primo file

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	909
N	907
P	903
Q	902
R	904
S	905
T	910

Buffer

00	B	903
01	A	901
	C	901
02		

Bucket

00

01

02

Primo file

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	909
N	907
P	903
Q	902
R	904
S	905
T	910

Buffer

00

B	903

01

A	901
C	901

02

Bucket

00

01

A	901
C	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

B	903

01

D	904

02

Bucket

00

01

A	901
C	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

B	903

01

D	904

02

E	911

Bucket

00

01

A	901
C	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

B	903
F	912

01

D	904

02

E	911

Bucket

00

01

A	901
C	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

01

D	904

02

E	911

Bucket

00

B	903
F	912

01

A	901
C	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

01

D	904

02

E	911
G	902

Bucket

00

B	903
F	912

01

A	901
C	901

02

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

01

D	904

02

Bucket

00

B	903
F	912

01

A	901
C	901

02

E	911
G	902

Primo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Buffer

00

01

02

Bucket

00

B	903
F	912

M	909
P	903

01

A	901
C	901

D	904
H	904

L	907
N	907

R	904
T	910

02

E	911
G	902

Q	902
S	905

Secondo file

A	901
B	903

C	901
D	904

E	911
F	912

G	902
H	904

L	907
M	909

N	907
P	903

Q	902
R	904

S	905
T	910

Bucket

00

B	903
F	912

M	909
P	903

01

A	901
C	901

D	904
H	904

L	907
N	907

R	904
T	910

02

E	911
G	902

Q	902
S	905

Bucket

00

01

02

Buffer

00

01

02

901	X
910	Y

918	X
911	Z

902	Z
912	Y

919	X
903	W

913	X
904	Y

920	V
914	V

905	X
915	Y

908	Z
916	Y

907	X
917	W

906	X
909	Y

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	909
N	907
P	903
Q	902
R	904
S	905
T	910

Bucket

00	B	903
	F	912
	M	909
	P	903
01	A	901
	C	901
	D	904
	H	904
	L	907
	N	907
	R	904
	T	910
02	E	911
	G	902
	Q	902
	S	905

Bucket

00	918	X
	912	Y
	903	W
	915	Y
	906	X
	909	Y
01	901	X
	910	Y
	919	X
	913	X
	904	Y
	916	Y
	907	X
02	911	Z
	902	Z
	920	V
	914	V
	905	X
	908	Z
	917	W

Buffer

00		
01		
02		

901	X
910	Y
918	X
911	Z
902	Z
912	Y
919	X
903	W
913	X
904	Y
920	V
914	V
905	X
915	Y
908	Z
916	Y
907	X
917	W
906	X
909	Y

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	909
N	907
P	903
Q	902
R	904
S	905
T	910

Bucket

00	B	903
	F	912
	M	909
	P	903
01	A	901
	C	901
	D	904
	H	904
	L	907
	N	907
	R	904
	T	910
02	E	911
	G	902
	Q	902
	S	905

Bucket

00	918	X
	912	Y
	903	W
	915	Y
	906	X
	909	Y
01	901	X
	910	Y
	919	X
	913	X
	904	Y
	916	Y
	907	X
02	911	Z
	902	Z
	920	V
	914	V
	905	X
	908	Z
	917	W

901	X
910	Y
918	X
911	Z
902	Z
912	Y
919	X
903	W
913	X
904	Y
920	V
914	V
905	X
915	Y
908	Z
916	Y
907	X
917	W
906	X
909	Y

A	901
B	903
C	901
D	904
E	911
F	912
G	902
H	904
L	907
M	909
N	907
P	903
Q	902
R	904
S	905
T	910

Bucket

00

B	903
F	912
M	909
P	903

01

A	901
C	901
D	904
H	904
L	907
N	907
R	904
T	910

02

E	911
G	902
Q	902
S	905

Bucket

00

918	X
912	Y
903	W
915	Y
906	X
909	Y

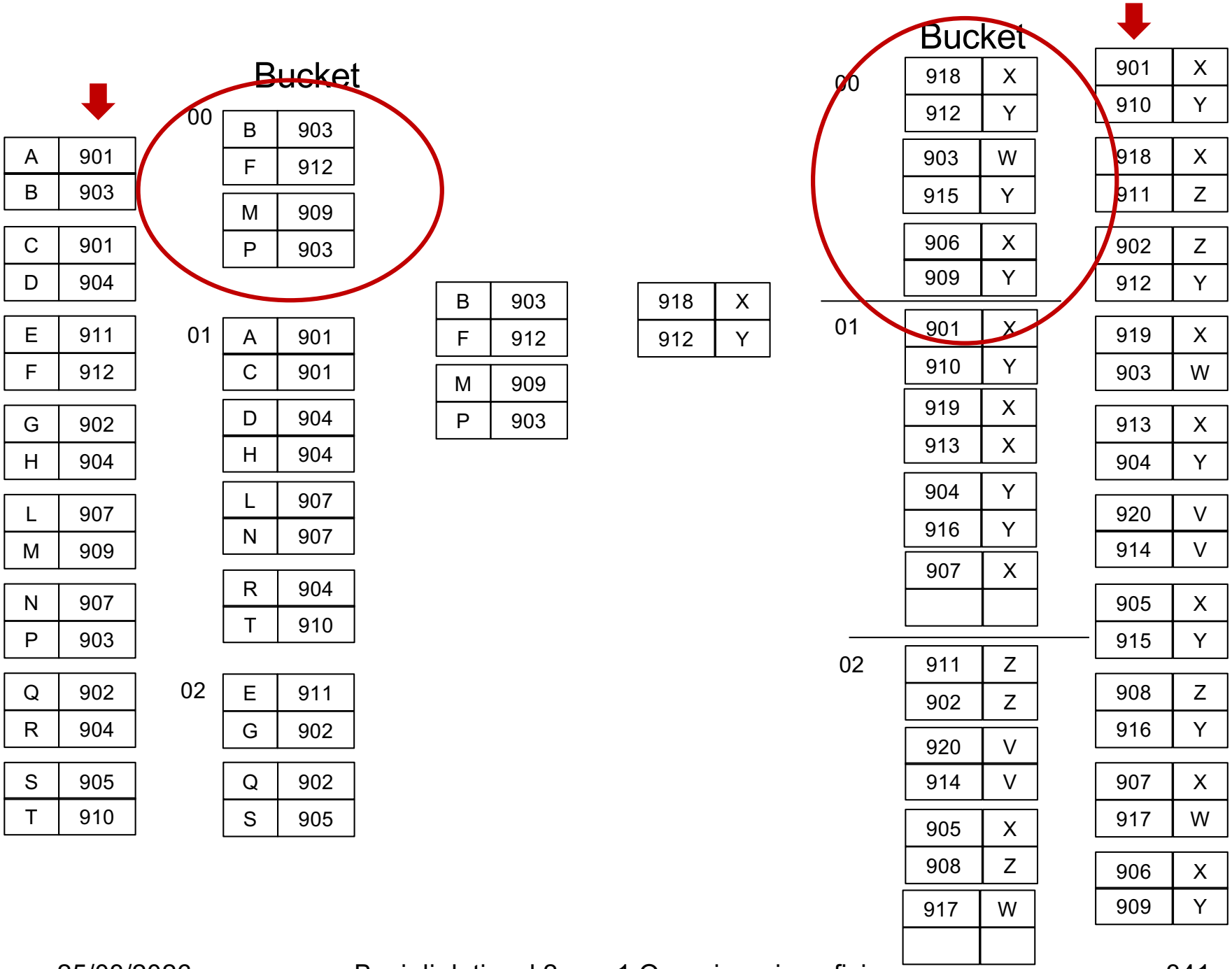
01

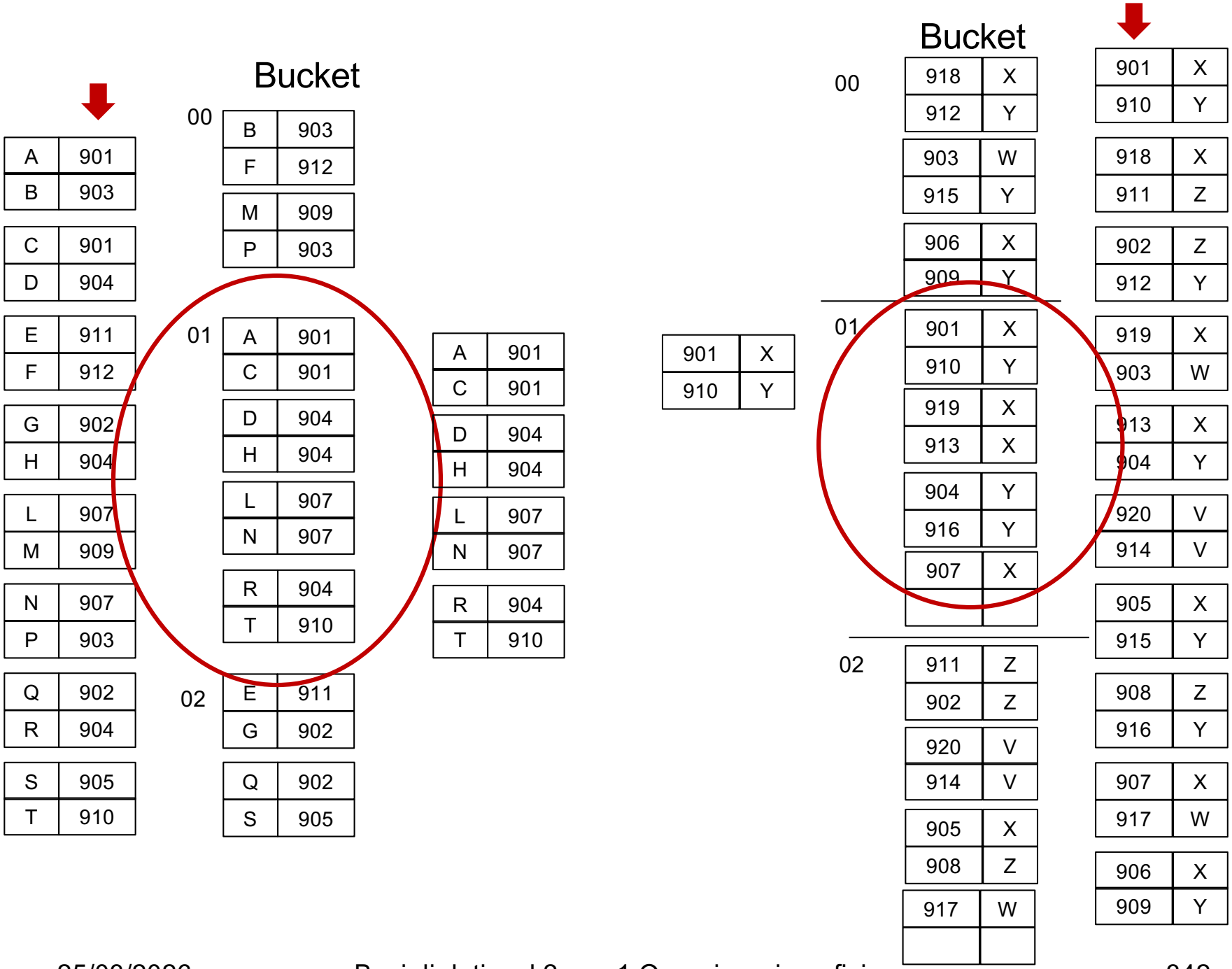
901	X
910	Y
919	X
913	X
904	Y
916	Y
907	X

02

911	Z
902	Z
920	V
914	V
905	X
908	Z
917	W

901	X
910	Y
918	X
911	Z
902	Z
912	Y
919	X
903	W
913	X
904	Y
920	V
914	V
905	X
915	Y
908	Z
916	Y
907	X
917	W
906	X
909	Y


Bucket
00
01
02
Bucket
00
01
02


Bucket
00
01
02
A 901
B 903
C 901
D 904
E 911
F 912
G 902
H 904
L 907
M 909
N 907
P 903
Q 902
R 904
S 905
T 910
B 903
F 912
M 909
P 903
A 901
C 901
D 904
H 904
L 907
N 907
R 904
T 910
E 911
G 902
Q 902
S 905
A 901
C 901
D 904
H 904
L 907
N 907
R 904
T 910
901 X
910 Y
Bucket
00
01
02
918 X
912 Y
903 W
915 Y
906 X
909 Y
901 X
910 Y
919 X
913 X
904 Y
916 Y
907 X
911 Z
902 Z
920 V
914 V
905 X
908 Z
917 W
901 X
910 Y
918 X
911 Z
902 Z
912 Y
919 X
903 W
913 X
904 Y
920 V
914 V
905 X
915 Y
908 Z
916 Y
907 X
917 W
906 X
909 Y

Hash-join con buffer, costi

- Sfruttando i buffer, si può usare una funzione hash con numero di valori diversi paragonabile al numero di pagine di buffer P a disposizione e **così si può spesso eseguire l'algoritmo in due passate di lettura più una scrittura**:
  - Ciascun file viene letto e riorganizzato in P liste di blocchi (costo Bi per la lettura e Bi per la riscrittura, numero di blocchi e non di record perché si scrivono blocchi completi) poi, per ciascun valore della funzione hash si confrontano le liste omologhe (costo B1 + B2 per la lettura).
  - L'algoritmo non richiede altri accessi se le singole partizioni di uno dei file entrano in memoria, cioè se min(B1,B2)/P < P cioè se P^2 > min(B1,B2). In tal caso il costo è 3(B1+B2) + il numero di blocchi del risultato (se va scritto)
  - Nota i calcoli sono approssimati, perché la funzione hash ha una distribuzione

Confronto costi

- Risultato piccolo (molta selettività):
  - spesso conviene nested loop con indice
- Operandi grandi di dimensione paragonabile:
  - Mergejoin (in teoria anche HashJoin, ma non è garantita la distribuzione uniforme dei valori)
- Operandi grandi, ma di dimensioni diverse:
  - Hashjoin (le porzioni della tabella più piccola possono entrare in memoria)